



MÒDUL 1 · SESSIÓ 2 · MF1 — EINES D'IA

OpenCode a fons: l'agent de codi

De l'editor AI-first a l'agent amb arnès: harness, MCP, skills i gestió del context

Eines d'IA de Desenvolupament i Programació · CTRD0040 · Formador: Armand Laplana

El paradigma AI-first



1. Context complet

El model veu tot el projecte, no només el fitxer actual.



2. Edició multi-fitxer

Canvis coordinats en diversos fitxers alhora.



3. Execució real

Comandes, tests i builds dins del projecte.



4. Iteració autònoma

Detecta errors i s'auto-corregeix.

El paradigma AI-first prioritza eines on l'LLM és el nucli de l'editor, no un plugin afegit. El desenvolupador passa a ser el DIRECTOR del procés.

Què és OpenCode

- **Agent de codi obert:** un agent d'IA de codi obert; alternativa a Claude Code.
- **Tres interfícies:** terminal, aplicació d'escriptori o extensió d'IDE.
- **Open-source:** i el més utilitzat de la seva categoria.
- **Agnòstic al model:** tu tries el proveïdor i el model que vols connectar.
- **Capa d'ús gratuïta:** es pot fer servir sense pagar (capa gratuïta o compte propi).

Instal·lació i comandes més utilitzades

Descàrrega: opencode.ai/download · Documentació: opencode.ai/docs

```
opencode           obre l'agent al projecte (terminal)
opencode web       interfície web local en lloc del terminal
/help             guia ràpida i catàleg de funcions
/connect          selecciona els proveïdors d'IA
/models           selecciona el model de cada proveïdor
/init             analitza el projecte i genera AGENTS.md
/agents o TAB     canvia entre mode Build i Plan
/new · /session · /status · exit    gestió de sessions
CTRL+P comandes  · CTRL+T nivell d'esforç del model
```

Els dos modes de l'agent

Plan Mode — quan usar-lo

- SÍ** Tasques grans
- SÍ** Refactors
- SÍ** Supervisió d'estratègia
- SÍ** Primer proposa el pla; executa quan dius OK

Plan Mode — quan NO

- NO** Tasques petites
- NO** Correccions òbvies
- NO** Prototipatge ràpid
- NO** Per a això: Build Mode (acció immediata)

Build Mode: actua directament sobre el codi. **Plan Mode:** planifica, ho revises i només executa amb el teu OK.

Arnès: guardarraïls per a l'agent

- **El problema:** l'indeterminisme dels LLM exigeix una estructura de control professional.
- **Què és un arnès:** el conjunt de «guardarraïls» que fan que l'agent treballi sota convencions i estàndards de producció.
- **AGENTS.md:** fitxers de text que actuen com a system prompts persistents del projecte; l'agent els consulta abans de cada acció.
- **Regla pràctica:** perquè no hi hagi soroll ni ineficiència, l'AGENTS.md no hauria de passar de 500 línies.

Variants segons l'eina: AGENT.md (genèric), CLAUDE.md, .cursorrules, copilot-instructions.md.

Què incloure a l'AGENTS.md

Vuit blocs recomanats (referència: agents.md):

Stack tecnològic

Convencions de codi

Patrons

Prohibicions

Estructura de projecte

Flux de treball

Testing, CI/CD

Estil de commits i PRs

Es genera fàcil amb /init, que analitza el projecte i en crea una primera versió. Després s'edita a mà.

Model Context Protocol: connectar eines

Què és / avantatges

- Interfície estàndard per connectar eines
- Estàndard obert i reutilitzable entre IDEs
- Seguretat per disseny
- Ecosistema creixent (API JSON-RPC)

A què s'hi connecta

- Dades i fitxers (PostgreSQL, SQLite, Drive)
- Eines de desenvolupament (Git, Sentry...)
- Eines de productivitat (Slack, Maps...)
- Documentació de llibreries (Context7)

On buscar servidors MCP

mcpservers.org

- Catàleg de servidors MCP de la comunitat
- Per categoria i popularitat
- Fitxes amb instruccions d'instal·lació
- El visitarem a classe

mcp.directory

- Un altre directori de MCPs disponibles
- Per comparar i descobrir opcions
- Útil per trobar el MCP que necessites
- El visitarem a classe

Context7: el MCP que configurarem a la pràctica

- **Què és:** un servidor MCP que dona a l'agent documentació i exemples reals des de les fonts oficials de cada llibreria.
- **Per a què el volem:** per al DISSENY de la nostra web: que l'agent faci servir HTML/CSS actuals i no APIs obsoletes.
- **Menys al·lucinacions:** informació específica per versió, sense sortir de l'editor.
- **A la pràctica:** el configurarem al NOSTRE projecte (web-apunts) i, en demanar millores de disseny, hi afegirem «...utilitza context7».

Coneixement intern vs. accions repetitives

Skills — coneixement intern

- Instruccions i bones pràctiques
- L'agent les llegeix sol abans d'actuar
- Fitxers SKILL.md (+ scripts)
- Es desen a .opencode/skills

Commands — accions que invoques

- Accions que executes tu amb «/»
- Ideals per a passos repetitius i estrictes
- Fitxers .md amb descripció i passos
- Es desen a .opencode/commands

Skills: coneixement a mida per a l'agent

- **Què és una skill:** un paquet de coneixement (instruccions i bones pràctiques) que l'agent llegeix SOL abans d'actuar; no l'has d'invocar.
- **Anatomia (SKILL.md):** frontmatter obligatori amb «name» (igual que la carpeta) i «description»; a sota, les instruccions. Pot dur scripts de suport.
- **On va:** a `.opencode/skills/<nom>/SKILL.md`. L'agent decideix quan li cal segons la «description».
- **On trobar-ne:** al directori `skills.sh` es descobreixen i s'instal·len amb «`npx skills add <owner/repo>`».
- **A la pràctica:** crearem la skill «estil-apunts» (paleta i to de la web) perquè l'agent mantingui l'estil sol.

Skill vs. Command: la skill és coneixement que l'agent consulta sol; el command és una acció que invoques tu amb «/». Doc: opencode.ai/docs/skills

Higiene de context: no malgastar tokens

- **Documentació de tasques:** una carpeta /docs on l'agent desa notes i llistes de tasques: «particiona el cervell» i recupera context sense saturar la finestra.
- **/compact:** resumeix l'historial crític i redueix l'ús de tokens sense perdre el fil; preferible a /new perquè preserva la continuïtat.
- **Per què importa:** passar d'un ús descontrolat de la IA a un desenvolupament professional i estructurat.

No deixis que l'agent consumeixi tokens sense criteri: aplica bones pràctiques d'higiene del context.

El bucle de verificació



(Repeteix) — el desenvolupador manté el control absolut i actua com a director del procés.

On anem: orquestrar amb especificacions

Spec-Driven Development

Escriure l'especificació abans del codi: el «contracte» que guia l'agent.

El cicle SDD

Constitució → Specify → Plan → Tasks → Implement → Verify.

Multiagents

Coordinador, implementador i verificador: separar responsabilitats.

Loop engineering

Dissenyar bucles on l'agent itera sol fins a complir l'objectiu.

Avui: l'agent i el seu arnès. Demà: com orquestrar-lo amb mètode.



L'agent, sota control

Mòdul 1 · Dia 2 — OpenCode, harness, MCP i skills · CTRD0040 · 60 h